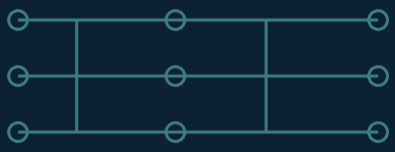
UPES / SCHOOL OF COMPUTER SCIENCE

ACADEMIC YEAR 2026–27

TECHNICAL PROJECT REPORT

BitNet-C

A Bitwise Optimized Binary Neural Network for Ultra Low Memory Activation



TEAM AMATEURS TEAM 11 | BATCH 10

PREPARED BY

Nimit · Team Lead & Lead Developer

Pranav Bhasin

Abhimanyu Jakhar

Aasma Sharda

SUBMITTED TO

Prof. Subhranil Das

First-Year B.Tech C Programming Mini Hackathon
University of Petroleum and Energy Studies, Dehradun

VERSION 2.4 / 2026

Project and Team Information

Project field	Details
Project title	BitNet-C: A Bitwise Optimized Binary Neural Network for Ultra Low Memory Activation
Institution	UPES, Dehradun
Programme	B.Tech Computer Science and Engineering
Event	First-Year C Programming Mini Hackathon
Team	Amateurs | Team 11 | Batch 10
Version	2.4
Faculty	Prof. Subhranil Das
Repository	github.com/realnimit1-byte/BitNet-C

Team roster

Name	SAP ID	Assigned role
Nimit	590033868	Team Lead & Lead Developer
Pranav Bhasin	590032768	Testing & Validation Coordinator
Abhimanyu Jakhar	590032062	Research & Analysis Coordinator
Aasma Sharda	590032474	Documentation & Presentation Coordinator

Role descriptions in Chapter 16 represent the proposed allocation of work; final contribution claims should reflect the activities actually completed by each member.

Acknowledgement

Team Amateurs expresses sincere gratitude to Prof. Subhranil Das for guidance and support during the C Programming Mini Hackathon. We thank the University of Petroleum and Energy Studies for providing the opportunity to apply fundamental C programming concepts to a compact neural-network prototype. We also acknowledge the contributions of the team members in project coordination, development, testing, analysis and documentation. The use of AI assistance is disclosed in Chapter 15.

Abstract

BitNet-C v2.4 is an educational binary neural-network prototype written in C. The network uses eight inputs, four hidden neurons and one output neuron. Binary weights are encoded in five bytes and the four hidden activations are packed into one byte. XNOR and population-count calculations replace conventional floating-point dot products for binary-valued inference. A bit-flip hill-climbing procedure with random restarts optimizes weights for a defined task: predicting whether an eight-bit input contains at least four set bits. The application includes a deterministic stratified training/test split, float32 parity checks, benchmarking, model file persistence and a terminal neural-network visualizer. Representation-level storage falls from 144 to five bytes for weights (96.53%) and from 16 to one byte for hidden activations (93.75%). A prior v2.2 evaluation reported 256/256 correct classifications; this finite synthetic result is not evidence of general-purpose machine-learning performance.

Table of Contents

Project and Team Information	2
Acknowledgement	3
Abstract	3
Table of Contents	4
1. Introduction	5
2. Problem Statement and Motivation	5
3. Objectives and Scope	5
4. Literature Review and Technical Background	5
5. Proposed Solution and System Architecture	6
6. Methodology and Mathematical Foundation	6
7. C Programming Implementation	6
8. Binary Weight Training and Optimization	7
9. Dataset Preparation and Classification	7
10. Memory Optimization and Storage Analysis	7
11. Performance Benchmarking	8
12. Model Persistence and File Handling	8
13. Testing, Validation and Sample Outputs	8
14. Results, Comparative Analysis and Discussion	8
15. AI-Assisted Development and Technical Enhancement	9
16. Team Roles and Individual Contributions	9
17. Challenges, Limitations and Lessons Learned	10
18. Future Scope and Enhancements	10
19. Conclusion	11
20. References	11
Appendix A — Algorithm Summary	11
Appendix B — Figures and Flowcharts	11
Appendix C — Repository and Submission Notes	11

1. Introduction

Overview

Binary neural networks restrict weights and activations to two possible values, commonly represented as -1 and +1. Their arithmetic can be expressed using packed bits, which makes them useful for demonstrating the relationship between data representation, memory consumption and inference algorithms.

Project context

BitNet-C is a first-year C programming Mini Hackathon project. Its goal is to build an understandable inference system without external machine-learning libraries.

2. Problem Statement and Motivation

Assigned problem statement

BitNet-C: A Bitwise Optimized Binary Neural Network for Ultra Low Memory Activation.

Motivation

Storing each weight as a 32-bit float can be wasteful when the model needs only two weight states. The project investigates compact weight and activation storage and the correctness of corresponding bitwise inference.

3. Objectives and Scope

Objectives

Implement an 8-4-1 binary neural network; compute XNOR/popcount dot products; optimize binary weights; evaluate a labeled eight-bit task; compare storage and execution time; persist trained weights; and provide a menu-driven visualization.

Scope

The system is an educational prototype for 256 possible eight-bit inputs, not a deployment-ready general-purpose AI model.

4. Literature Review and Technical Background

Binary inference

XNOR identifies matching binary signs; population count counts the matches. The binary dot product is 2 x matches - number of bits. This identity follows because a matching sign contributes +1 and a mismatching sign contributes -1.

Related work

BinaryConnect (Courbariaux et al., 2015) and Binarized Neural Networks (Hubara et al., 2016) establish the broader research context for binary weights and activations. BitNet-C is a small independent teaching prototype, not a reproduction of a production BitNet model.

5. Proposed Solution and System Architecture

Layer structure

Eight input bits feed four hidden neurons, whose sign activations are packed into one byte. A four-input output neuron produces a +1 or -1 prediction, mapped to Class 1 or Class 0.

Data flow

Input validation -> bitwise hidden inference -> packed hidden activations -> output inference -> prediction and optional comparison against a reference label.

6. Methodology and Mathematical Foundation

XNOR/popcount dot product

For N binary sign values, dot(A,B) = 2 x popcount(XNOR(A,B) AND mask) - N. A mask limits the operation to the valid input bits.

Activation

sign(x) returns +1 when x is nonnegative and -1 otherwise. Positive activations are stored as one bits; negative activations as zero bits.

Reference label

The expected class is 1 if the input contains at least four one bits; otherwise it is 0. This label-generation rule is used for training and evaluation, not as a shortcut inside binary inference.

7. C Programming Implementation

Core functions

count_bits(), binary_dot_product(), binary_activation(), binary_network() and float_network() implement the central inference and reference operations. Input parsing validates exactly eight binary digits.

Language features

The program demonstrates uint8_t and uint32_t, arrays, functions, loops, conditionals, pointers, bitwise XOR/NOT/AND/shifts, formatted output and binary file I/O.

Interactive menu

The twelve-option menu supports recognition, memory comparison, benchmarking, parity verification, architecture, training, accuracy, dataset details, saving, loading, visualization and exit.

8. Binary Weight Training and Optimization

Optimization approach

A deterministic bit-flip hill-climbing search tests changes to individual hidden and output weight bits. A change is retained only if training accuracy improves. Multiple random restarts explore different initial weight configurations.

Training integrity

The search score uses only the training partition. Held-out test inputs are not used to select candidate weights. Training is discrete search, not gradient-based backpropagation.

9. Dataset Preparation and Classification

Dataset

All integers from 0 through 255 are treated as eight-bit patterns and labeled according to their number of set bits. There are 93 Class 0 and 163 Class 1 examples.

Partition

A deterministic shuffled stratified holdout assigns 70 Class 0 and 122 Class 1 patterns to training, and 23 Class 0 and 41 Class 1 patterns to testing.

Evaluation

Training and held-out test accuracies are reported separately. A confusion matrix summarizes true positives, true negatives, false positives and false negatives on the test set.

10. Memory Optimization and Storage Analysis

Weight storage

The network has 8 x 4 + 4 x 1 = 36 binary weights. Four eight-bit hidden weight containers and one eight-bit output weight container occupy five bytes, compared with 36 x 4 = 144 bytes for float32 weights.

Activation storage

Four float32 hidden activations occupy 16 bytes; four binary activation bits fit into one uint8_t byte. These are representation-level storage figures, not measurements of total process RAM.

Storage component	Float32	Binary	Reduction
36 weights	144 bytes	5 bytes	96.53%
4 hidden activations	16 bytes	1 byte	93.75%

11. Performance Benchmarking

Method

The program times 1,000,000 inference iterations of the binary and float32 reference networks using clock(). A volatile checksum helps preserve work during benchmarking.

Interpretation

The reported ratio is float32 execution time divided by binary execution time. Actual timings depend on compiler options and execution environment. Measured benchmark numbers should be inserted from the final run; no speedup is assumed.

12. Model Persistence and File Handling

File format

bitnet_model.dat contains a four-byte BNC1 signature, four hidden-weight bytes and one output-weight byte: nine bytes total.

Validation

The loader rejects a missing file, invalid signature, malformed length or output weight outside the four-bit range. OnlineGDB persistence across separate sessions should be checked before relying on it in a live demonstration.

13. Testing, Validation and Sample Outputs

Functional tests

Observed v2.4 screenshots confirm Class 0 for 00000000 and 00000111, Class 1 for 11111111 and 10110110, and rejection of 10112010 as invalid binary input.

Correctness checks

A parity check compares binary and float32 inference over all 256 inputs. The final v2.4 output should be captured for the submission record.

Evidence

The repository TEST_CASES.md and screenshots/ directory contain detailed test descriptions and visual evidence. Additional pending cases must not be marked passed without execution.

14. Results, Comparative Analysis and Discussion

Observed development results

An early predefined-weight model achieved 157/256 (61.33%) overall classification accuracy. A later v2.2 optimized run reported 192/192 training, 64/64 held-out test and 256/256 total correct predictions.

Interpretation

The improvement illustrates that weight selection matters even for a small binary network. Because all 256 possible inputs are enumerable and labels are synthetic, the result does not establish real-world generalization.

Storage results

Weight storage reduction is 96.53%; hidden activation storage reduction is 93.75%. These savings do not include code, stack, datasets or operating-system overhead.

Metric	Result
Original baseline	157/256 (61.33%)
v2.2 optimized training	192/192 (100%)
v2.2 optimized held-out test	64/64 (100%)
v2.2 overall	256/256 (100%)

15. AI-Assisted Development and Technical Enhancement

Role of AI

AI assistance was used for generating and refining C code, suggesting algorithmic improvements, troubleshooting discrepancies, interpreting tests, and drafting technical documentation. The team directed the workflow, ran the program, inspected outputs and selected the final features.

Verification and accountability

AI suggestions were not treated as automatically correct. For example, predicted hidden activations and output values were checked against observed execution, and validation design was revised after identifying a biased split. The report acknowledges substantive AI assistance rather than presenting it as only minor proofreading.

16. Team Roles and Individual Contributions

Nimit — Team Lead & Lead Developer

Responsible for technical direction, architecture, integrating C components, coordinating debugging and testing, and maintaining the repository. Final statements should reflect completed work.

Pranav Bhasin — Testing & Validation Coordinator

Assigned to execute input test cases, compare expected and actual results, and organize evidence for verification.

Abhimanyu Jakhar — Research & Analysis Coordinator

Assigned to explain the problem motivation, research binary neural-network concepts, and interpret memory comparisons.

Aasma Sharda — Documentation & Presentation Coordinator

Assigned to organize technical documentation, communicate program workflow, and prepare presentation materials.

Contribution integrity

These are assigned role descriptions; each member should review and confirm the work they personally completed before submission.

Member	SAP ID
Nimit	590033868
Pranav Bhasin	590032768
Abhimanyu Jakhar	590032062
Aasma Sharda	590032474

17. Challenges, Limitations and Lessons Learned

Challenges

Important development challenges included validating bit ordering, distinguishing implementation parity from task accuracy, improving the training/test split and documenting representation-level memory savings accurately.

Limitations

The model handles only eight-bit inputs and one synthetic task; training is simple hill climbing; timing is environment-dependent; and the stored model file may not persist across all online IDE sessions.

18. Future Scope and Enhancements

Planned enhancements

Potential improvements include larger packed networks, broader labeled datasets, repeated benchmark statistics, hardware popcount support, model portability checks and deployment on constrained hardware.

19. Conclusion

Summary

BitNet-C v2.4 demonstrates a complete small binary inference pipeline in standard C, including training, evaluation, compact representation, persistence and visualization. The project combines fundamental programming concepts with a concrete systems-oriented optimization problem.

20. References

Selected references

[1] Courbariaux, Bengio and David, BinaryConnect: Training Deep Neural Networks with binary weights during propagations (2015).
[2] Hubara et al., Binarized Neural Networks (2016).
[3] ISO/IEC 9899:2011, Programming Languages — C.
[4] Project repository: github.com/realnimit1-byte/BitNet-C.

Appendix A — Algorithm Summary

Core inference pseudocode

Validate 8-bit input; for each hidden neuron compute XNOR/popcount dot product and sign; pack four signs; compute output dot product and sign; map sign to class.

Training pseudocode

For each restart: initialize weights; repeatedly flip individual bits; retain only changes that improve training score; restore the best training-scoring weights.

Appendix B — Figures and Flowcharts

Diagram references

See flowchart.png in the GitHub repository for the overall program diagram. The report architecture is 8 inputs -> 4 hidden neurons -> 1 output neuron.

Appendix C — Repository and Submission Notes

Repository

github.com/realnimit1-byte/BitNet-C

Required submission materials

main.c, README.md, ALGORITHM.md, TEST_CASES.md, flowchart.png, screenshots/ and the presentation file when completed.